Lesson 9

Tools, Knowledge Memory and HITL

Agenda

Tools & Process

- Tools
  Characteristics of Tools
  Built-in Tools
  Custom Tools
  - Using @tool
  - Subclassing BaseTool
- Hierarchical Process
- Sequential vs Hierarchical

Memory, Knowledge & HITL

- Knowledge to Agents and Crew
  Text Sources (Raw string, .txt, .pdf)
  Structured Data (csv, excel, json)
  Web Content
  - CrewDocling
  - WebSiteSearch
- Memory
- Human in the Loop

Tools in CrewAI

Capabilities that empower agents to act

What is a Tool?

In CrewAI, a tool is a capability or function that agents use to perform tasks — from simple searches to complex interactions — while supporting smooth teamwork.

🔍 Web Searching

📊 Data Analysis

📝 Content Generation

🤝 Agent Collaboration

- Tools bridge the gap between LLM reasoning and real-world actions
- Agents decide when and how to use tools based on task requirements
- Tools can be built-in or custom-built for specialized needs

Key Characteristics of Tools (1/2)

Utility

Crafted for tasks such as web searching, data analysis, content generation, and agent collaboration.

Integration

Boosts agent capabilities by seamlessly integrating tools into their workflow.

Customizability

Provides flexibility to develop custom tools or utilize existing ones, catering to specific agent needs.

Key Characteristics of Tools (2/2)

Error Handling

Incorporates robust error handling mechanisms to ensure smooth operation and resilience.

Caching Mechanism

Features intelligent caching to optimize performance and reduce redundant operations — agents efficiently reuse previously obtained results.

Async Support

Supports asynchronous tools perform non-blocking operations like network requests, file I/O without blocking the main execution thread.

Built-in Tools

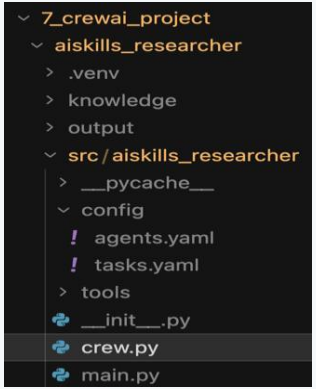
CrewAI provides built-in tools through the crewai[tools] package.

Install: `pip install 'crewai[tools]'` or `uv pip install "crewai[tools]"`

Documentation: https://docs.crewai.com/en/concepts/tools#available-crewai-tools

Demo Folder: 7_crewai_project(Tools & Context)/aiskills_researcher

Project Structure:



The aiskills_researcher project uses SerperDevTool for live web search in a 3-agent pipeline: Researcher → Analyst → Report Writer.

SerperDevTool — Web Search

About this Code

Defines a CrewAI sequential workflow to produce an AI Skills Report (2026) using three agents:

Researcher

Uses SerperDevTool to search the web and generate research notes with sources.

Analyst

Turns the research into structured categories and key insights.

Report Writer

Creates a user-friendly final report saved to output/ai_skills_2026.md.

Setup SerperDevTool:

```
# 1. Install
pip install 'crewai[tools]'

# 2. Get free API key at serper.dev
# 3. Set environment variable
export SERPER_API_KEY="your_key_here"

# 4. Import and use
from crewai_tools import SerperDevTool
tool = SerperDevTool()
```

Key Concepts:

- Tasks run in order: research → analysis → report
- Tasks can reuse earlier outputs via context
- The report_writer writes based on what the researcher already found
- run() kicks off the crew, prints result, saves to output/ai_skills_2026.md

Custom Tools

Two main approaches to creating your own tools

Custom Tools — Two Approaches

① @tool Decorator

When to use:

Quick and simple tools with minimal setup.

Key Points

- Simplest approach: decorate a function
- Function docstring becomes the tool description
- Tool name is passed as a decorator argument

② Subclassing BaseTool

When to use:

Complex tools that need input validation, custom schemas, or more control.

Key Points

- Define an input schema using Pydantic (validation)
- More explicit control over name, description, arguments
- Best for tools with multiple/complex parameters

Custom Tool — @tool Decorator

Template

```
from crewai.tools import tool

@tool("Tool Name")
def my_tool(argument: str) -> str:
    """Description of what this tool does -
    agents use this to decide when to use
it."""
    # Your logic here
    return "result"
```

Example — Get Weather

```
import requests
from crewai.tools import tool

@tool("Get Weather")
def get_weather(city: str) -> str:
    """Fetch current weather for a given
    city from wttr.in."""
    endpoint = "https://wttr.in"
    response = requests.get(
        f"{endpoint}/{city}"
    )
    return response.text
```

Points to remember:

- The @tool decorator converts a regular function into a CrewAI-compatible tool
- The string argument to @tool (e.g., "Get Weather") becomes the tool's name shown to the LLM
- The docstring is used as the tool description — write it clearly so agents know when to call it

Custom Tool — Subclassing BaseTool

Template

```python
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field

class MyToolInput(BaseModel):
    """Input schema for the tool."""
    argument: str = Field(
        ..., description="Description"
    )

class MyCustomTool(BaseTool):
    name: str = "Tool Name"
    description: str = "What this tool does"
    args_schema: Type[BaseModel] = MyToolInput

    def _run(self, argument: str) -> str:
        # Your logic here
        return "result"
```

Example — WeatherTool

```python
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import requests

class WeatherInput(BaseModel):
    """Input schema for WeatherTool."""
    city: str = Field(
        ..., description="City name"
    )

class WeatherTool(BaseTool):
    name: str = "Get Weather"
    description: str = "Fetch weather."
    args_schema: Type[BaseModel] = WeatherInput

    def _run(self, city: str) -> str:
        endpoint = "https://wttr.in"
        response = requests.get(
            f"{endpoint}/{city}"
        )
        return response.text
```

How Tool Execution Works

1

Tool Assignment

Pass the tool in the agent's tools list: tools=[MyCustomTool()]

2

Task Execution

LLM decides if and when to call the tool based on task description and the tool's name/description.

3

Tool Invocation

The LLM provides arguments matching your args_schema. CrewAI calls _run() with those arguments.

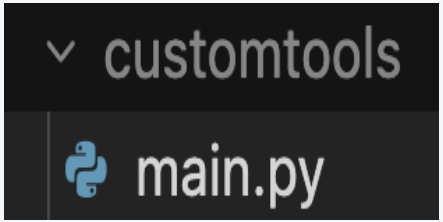
4

Result Handling

The string output is returned to the agent, which reflects on it and may use it to formulate its final answer.

Demo: Custom Tools in Action

Demo Folder: 7_crewai_project · No YAML config — agents and tasks defined directly in code



About the Demo

- Tool 1 (decorator): get_weather(city) — calls wttr.in
- Tool 2 (BaseTool): StockPriceTool — mock stock price for AAPL
- Weather Agent handles "Get weather for New York"
- Stock Agent handles "Get AAPL stock price"
- crew.kickoff() runs weather task first, then stock task

Visual Flow of Tool Execution:

```
Agent receives task
    ↓
Agent analyzes task → Determines tool is needed
    ↓
Agent selects tool (based on name & description)
    ↓
Agent prepares tool input parameters
    ↓
Input validated against args_schema (Pydantic)
    ↓
Tool._run() method executes
    ↓
Tool returns result
    ↓
Agent interprets result
    ↓
Agent decides:
    ├→ Use result to complete task → Task finished
    └→ Need more information → Call another tool or continue reasoning
```

Hierarchical Process

Manager-driven task coordination and delegation

Hierarchical Process — Overview

In a hierarchical process, you typically do NOT create a manager agent yourself (unless you want a custom one). CrewAI automatically creates a manager that:

1 Coordinates the Workflow

Decides which agent handles each task based on roles and capabilities.

2 Delegates Tasks

Assigns work dynamically based on agent roles and capabilities.

3 Validates Outcomes

Reviews results before moving forward to the next stage.

Default manager_llm

When you want a fast and simple hierarchical setup — CrewAI auto-creates the manager using the LLM you specify.

Custom manager_agent

When you need stronger control, clarity, and workflow customization — define and pass your own manager agent.

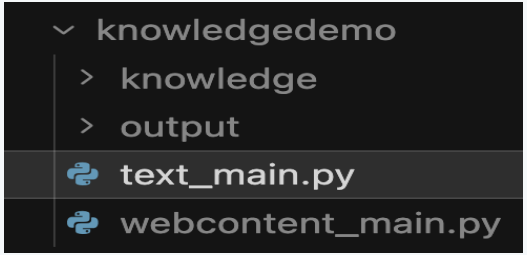
Hierarchical Process — manager_llm

```
crew = Crew(
    agents=[weather_agent, stock_agent],
    tasks=[weather_task, stock_task, report_task],
    process=Process.hierarchical,
    manager_llm="gpt-4o",   # Manager is auto-created using this LLM
    verbose=True
)
```

What Happens:

- The manager LLM automatically acts as the coordinator for the crew
- It decides which agent should handle each task and manages the task flow
- It collects and organizes the agent outputs to help produce the final result

Demo Screenshot:



Hierarchical Process — Custom Manager

```
manager = Agent(
    role="Student Support Manager",
    goal="Understand the student query and delegate it to the most
suitable specialist agent.",
    backstory=(
        "You are an academic support coordinator. "
        "Your job is to review the student's request, decide which
specialist is the best fit, "
        "and ensure the final response is relevant and focused."
    ),
    verbose=True,
    allow_delegation=True
)

student_support_crew = Crew(
    agents=[tutor_agent, study_planner_agent, quiz_creator_agent],
    tasks=[explain_task, study_plan_task, quiz_task],
    process=Process.hierarchical,
    manager_agent=manager,
    planning=True,
    verbose=True
)
```

Files: hierarchicalprocess > main1.py (manager_llm) · main2.py (custom manager)

What Happens:

- Manager agent reads the student query and understands what kind of help is needed
- Delegates to the most suitable specialist: tutor, study planner, or quiz creator
- Coordinates overall flow so the final response is relevant, focused, and well organized

Demo — main2.py:

Sequential vs Hierarchical

Feature	Sequential	Hierarchical
Task Order	Fixed order	Manager decides task allocation
Assignment	Pre-assigned to agents	Dynamic delegation based on capabilities
Nature	Simple, predictable	Flexible, adaptive

Sequential

Best for straightforward, predictable pipelines where task order is known upfront and each step feeds cleanly into the next.

Hierarchical

Best for complex, dynamic workflows where intelligent routing and adaptive task delegation are required.

Knowledge in CrewAI

Giving agents a reference library to consult while working

Knowledge in CrewAI — Overview

Knowledge in CrewAI allows agents to access and use external information sources during tasks. Think of it as giving agents a reference library they can consult while working.

Key Benefits:

- 📚 Enhance agents with domain-specific information
- 📊 Support decisions with real-world data
- 💾 Maintain context across conversations (ChromaDB persistence)
- ✅ Ground responses in factual information

Knowledge Sources Supported

Text Sources

- Raw strings
- Text files (.txt)
- PDF documents

Structured Data

- CSV files
- Excel spreadsheets
- JSON documents

Web Content

- CrewDoclingSource (URLs as knowledge)
- WebsiteSearchTool (semantic RAG search)
- Custom Knowledge Sources (extend BaseKnowledgeSource)

Behind the Scenes: RAG Pipeline

Ingest

1 Each source type has its own parser (PDF, CSV, web scraper) that extracts raw text.

Chunk

2 Extracted text is split into smaller, manageable pieces.

Embed

3 Each chunk is converted into a numerical vector using an embedding model.

Store

4 Vectors are stored in ChromaDB (bundled, zero config — like SQLite).

Retrieve

5 Agent's query is semantically matched against stored vectors — only the most relevant chunks returned.

Augment

6 Retrieved chunks are injected into the agent's prompt for accurate responses.

Generate

7 Retrieved information is used by the agent's LLM to produce its response.

Note: Only the ingestion step differs across source types — the rest of the pipeline is identical for all knowledge sources.

Knowledge — Key Points

🚫 Not regular file reading

CrewAI never passes entire files to the LLM — only the most relevant chunks are retrieved and injected.

🔄 Uniform pipeline

Only the ingestion step differs across source types; chunk → embed → store → retrieve is identical.

📦 ChromaDB included

ChromaDB is bundled with CrewAI — no separate installation required. Zero configuration needed.

🔀 Two RAG approaches

Implicit → knowledge_sources on Crew/Agent (ChromaDB managed automatically). Explicit → get_rag_client() for direct control.

Built-in Knowledge Parsers

Built-in Parsers — No Extra Install Required

Each source type has its own dedicated class under crewai.knowledge.source:

Source Type	Parser Class	Import Path
Raw strings	`StringKnowledgeSource`	`crewai.knowledge.source.string_knowledge_source`
Text files	`TextFileKnowledgeSource`	`crewai.knowledge.source.text_file_knowledge_source`
PDF	`PDFKnowledgeSource`	`crewai.knowledge.source.pdf_knowledge_source`
CSV	`CSVKnowledgeSource`	`crewai.knowledge.source.csv_knowledge_source`
Excel	`ExcelKnowledgeSource`	`crewai.knowledge.source.excel_knowledge_source`
JSON	`JSONKnowledgeSource`	`crewai.knowledge.source.json_knowledge_source`

External Library — Docling (requires: uv add docling)

`CrewDoclingSource` · `crewai.knowledge.source.crew_docling_source` · Web content / URLs

Crew-Level vs Agent-Level Knowledge

Crew-Level Knowledge

- Shared by all agents
- Set via Crew(knowledge_sources=[…])
- Good for policies and general information
- All agents access the same knowledge base

Agent-Level Knowledge

- Used only by a specific agent
- Set via Agent(knowledge_sources=[...])
- Good for specialized, role-specific info
- Each agent can have unique knowledge sources

Crew-Level Example:

```
crew = Crew(
    agents=[agent1, agent2],
    tasks=[task1, task2],
    knowledge_sources=[string_source]
)
```

Agent-Level Example:

```
agent = Agent(
    role="Document Analyst",
    goal="Analyze documents",
    backstory="Expert in analysis",
    knowledge_sources=[text_source, pdf_source]
)
```

Structured Data Knowledge Sources

1 CSV Files

```
from crewai.knowledge.source
  .csv_knowledge_source import
  CSVKnowledgeSource

csv_source =
CSVKnowledgeSource(
    file_paths=[
        "data.csv",
        "records.csv"
    ]
)
```

2 Excel Spreadsheets

```
from crewai.knowledge.source
  .excel_knowledge_source
import
  ExcelKnowledgeSource

# Supports multi-tab files
excel_source =
ExcelKnowledgeSource(
    file_paths=[
        "spreadsheet.xlsx",
        "report.xlsx"
    ]
)
```

3 JSON Documents

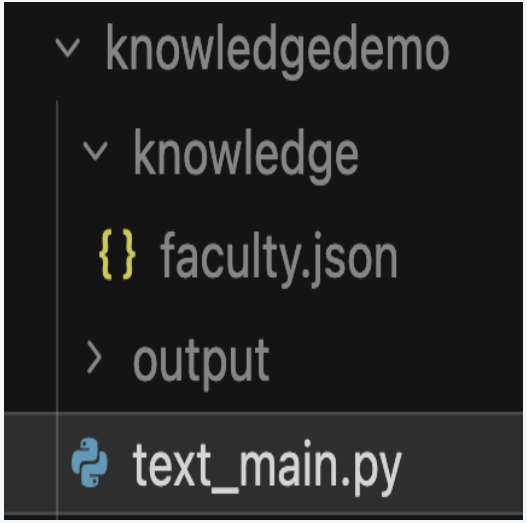
```
from crewai.knowledge.source
  .json_knowledge_source
import
  JSONKnowledgeSource

json_source =
JSONKnowledgeSource(
    file_paths=[
        "data.json",
        "config.json"
    ]
)
```

Place your files under the knowledge/ folder in your project root. · Demo: 7_crewai_project · See Slide 28 for screenshot.

Structured Data Demo — txt & json Knowledge

Demo: 7_crewai_project — text_main.py



About Demo:

- Hierarchical CrewAI Q&A crew with two agents
- Agent 1: answers from department text knowledge (.txt)
- Agent 2: answers from faculty JSON data (.json)
- A manager LLM decides which agent to use for each question
- Runs two queries: who teaches a subject + data science track info
- Prints combined results from both agents

Web Content — CrewDoclingSource

A knowledge source that uses the Docling library to extract content from web URLs and make it available as crew-wide knowledge.

Purpose

Pre-load web content into a knowledge base for RAG queries.

Best For

Static content · Known URLs upfront · Faster queries because content is already indexed.

How It Works:

- Fetches and processes web pages during initialization
- Stores content in a vector database (embeddings)
- Agents query the pre-loaded knowledge during execution

Requires: uv pip install docling

```
from crewai.knowledge.source.crew_docling_source import
CrewDoclingSource

content_source = CrewDoclingSource(
    file_paths=[
        "https://lilianweng.github.io/posts/2024-11-28-
reward-hacking",
    ],
)

crew = Crew(
    agents=[agent],
    tasks=[task],
    knowledge_sources=[content_source],
)
```

Web Content — WebsiteSearchTool

A RAG tool agents use during execution for semantic searches inside website content (on-demand searching).

```
from crewai_tools import WebsiteSearchTool

tool =
WebsiteSearchTool(website='https://example.com')

agent = Agent(
    role="Researcher",
    tools=[tool],
)
```

Key Difference

CrewDoclingSource:

Loads and indexes content before execution

WebsiteSearchTool:

Searches content during agent execution

Comparison: CrewDoclingSource vs WebsiteSearchTool

Aspect	CrewDoclingSource	WebsiteSearchTool
Type	Knowledge Source	RAG Tool
Content loaded	Before crew execution (pre-loaded)	During agent execution (on-demand)
Scope	Crew-wide	Agent-specific

When to Use Each — Web Content Options

Agents need consistent access to specific documents

→ **CrewDoclingSource**

Agent needs to search websites dynamically

→ **WebsiteSearchTool**

Content is known beforehand

→ **CrewDoclingSource**

Content URLs determined during execution

→ **WebsiteSearchTool**

Building a crew-wide knowledge base

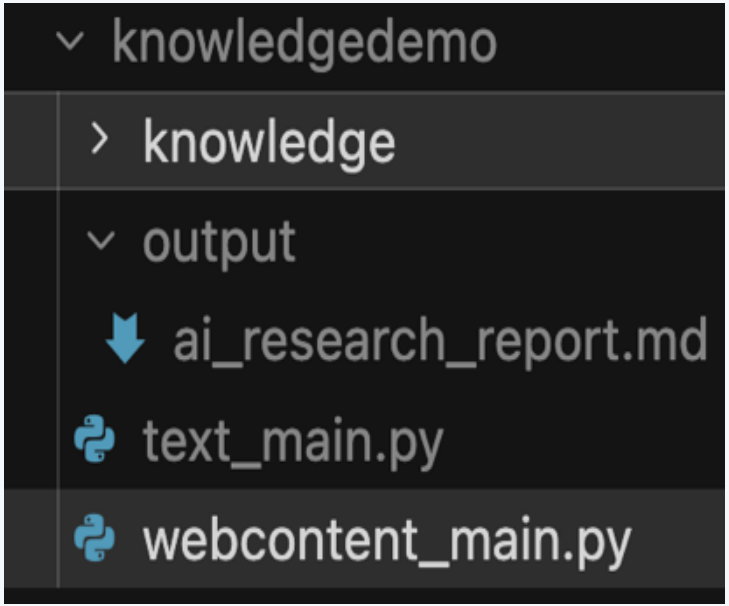
→ **CrewDoclingSource**

Performing ad-hoc web research

→ **WebsiteSearchTool**

Web Content Demo — webcontent_main.py

Demo: 7_crewai_project · webcontent_main.py



About webcontent_main.py:

- 2-step CrewAI pipeline reading two AI blog/paper-style webpages into a local knowledge store
- Knowledge loading: CrewDoclingSource ingests two provided URLs (reward hacking + hallucination)
- Agent 1 (Researcher): analyzes loaded content for given topic — extracts key concepts, methods, findings, and limitations
- Agent 2 (Writer): uses researcher's output as context to write a polished executive summary report + recommendations
- Saves output to output/ai_research_report.md
- Execution: runs tasks sequentially via crew.kickoff(...) and prints the final raw result

CrewAI Memory

Store, recall, and reuse information across tasks and runs

CrewAI Memory — Overview

Setting memory=True at the crew level enables shared memory for the whole crew. CrewAI automatically creates a default Memory() object, stores useful facts after tasks, and recalls relevant context before later tasks.

Shared Crew Memory

All agents share the same crew memory unless a specific agent is given its own. Supports collaboration and consistency.

Agent-Level Memory

Useful when you want private or scoped memory for one agent — for privacy or specialization.

LanceDB Storage

Data is stored locally using embedded vector database LanceDB, usually at ./.crewai/memory, persisting across runs.

For each task:

- Before execution — relevant memory is recalled and added to the prompt
- After execution — useful facts are extracted from the output and saved
- During saving — CrewAI uses embeddings for semantic search, checks for duplicates, stores memories in the background
- During recall — memories are ranked using semantic similarity, recency, and importance

Custom Memory Configuration

Instead of just using memory=True, you can create a custom Memory() object for more control over recall and storage behavior.

```
from crewai import Crew, Memory

memory = Memory(
    semantic_weight=0.4,
    recency_weight=0.4,
    importance_weight=0.2,
    recency_half_life_days=14,
)

crew = Crew(
    agents=[...],
    tasks=[...],
    memory=memory,
)
```

Important Parameters:

`semantic_weight`	Priority to meaning similarity
`recency_weight`	Priority to newer memories
`importance_weight`	Priority to more important facts
`recency_half_life_days`	Controls how fast old memories lose value
`consolidation_threshold`	Checks similar memories for duplication
`confidence_threshold_high/low`	Controls shallow vs deeper recall
`exploration_budget`	Sets extra exploration rounds in deep recall

Ref: https://docs.crewai.com/en/concepts/memory

Memory — Recall Modes & Summary

Shallow Recall

Fast vector search with no LLM calls. Efficient for most cases where straightforward semantic similarity is sufficient.

Deep Recall

Multi-step intelligent retrieval with LLM analysis, confidence routing, and optional exploration rounds. Used for complex or ambiguous queries.

Summary

CrewAI memory allows agents to store, recall, and reuse useful information across tasks and runs. Shared crew memory supports collaboration, while agent-level memory supports privacy or specialization. Default memory works automatically, and custom memory gives more control when needed.

Human-In-The-Loop (HITL)

Combining AI with human expertise to enhance decision-making

Human-In-The-Loop — Overview

Human-In-The-Loop (HITL) combines AI with human expertise to enhance decision-making and improve task outcomes.

Common Use Cases:

- Quality assurance and validation
- Complex decision-making scenarios
- Sensitive or high-stakes operations
- Creative tasks requiring human judgment
- Compliance and regulatory reviews

Best Practices:

- Be Specific: Provide clear, actionable feedback
- Stay Relevant: Only include information that helps improve execution
- Be Timely: Respond to HITL prompts promptly to avoid delays
- Review Carefully: Double-check feedback before submitting

Implementation — just set human_input=True on any task where you want human review:

Demo reference:

```
task = Task(
    description="Write a blog post about AI trends",
    agent=writer_agent,
    human_input=True    # Pauses for human review
)
```

HITL — Runtime Flow & Console Screenshots

Step	Agent	What Happens	Human Involved?
1	Researcher	Gathers AI trends	✅ You review the research
2	Editor	Fact-checks & cleans up	❌ Automatic
3	Writer	Writes the blog post	✅ You approve final draft

Core pattern: put human_input=True only on tasks where human judgment adds the most value — not every step needs review.

```
💬 Human Feedback Required

Provide feedback on the Final Result above.

• If you are happy with the result, simply hit Enter without typing
anything.
• Otherwise, provide specific improvement requests.
• You can provide multiple rounds of feedback until satisfied.
```

Lesson 9 — Summary

What We Covered

🛠 Tools

Built-in tools via crewai[tools], @tool decorator, and BaseTool subclassing

🔀 Hierarchical Process

Auto manager_llm or custom manager_agent for flexible task delegation

📚 Knowledge Sources

Text, Structured Data, Web Content all processed via the same RAG pipeline

🧠 Memory

Shared crew memory and custom Memory() for persistent, context-aware agents

👤 Human-In-The-Loop

human_input=True for targeted human review at critical task checkpoints

ternal Confidential